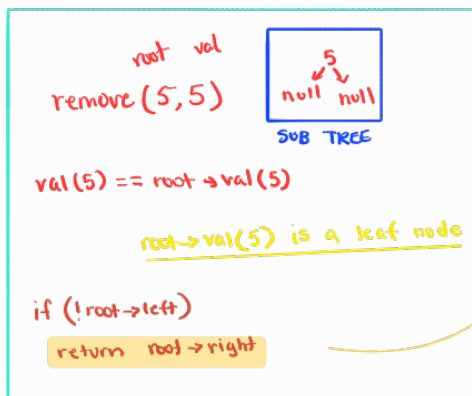
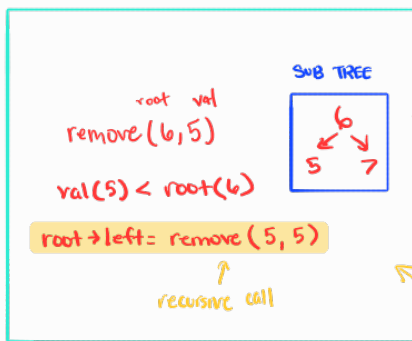
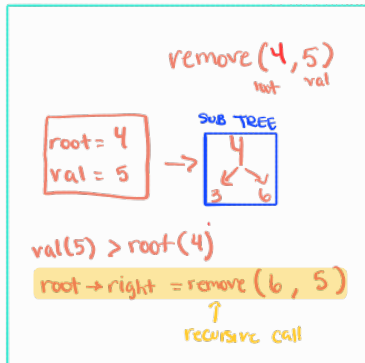
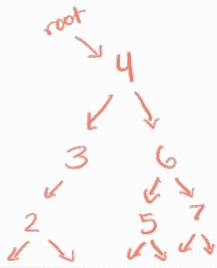


REMOVE LEAF NODE



return root



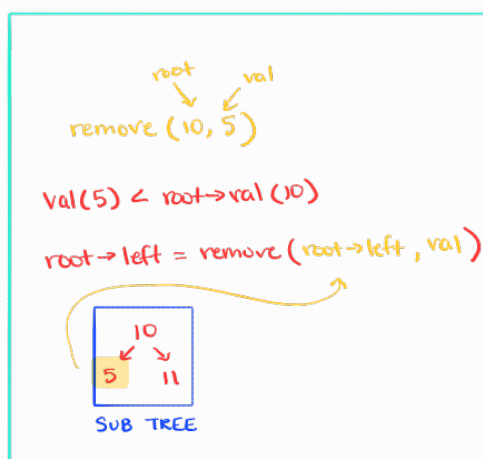
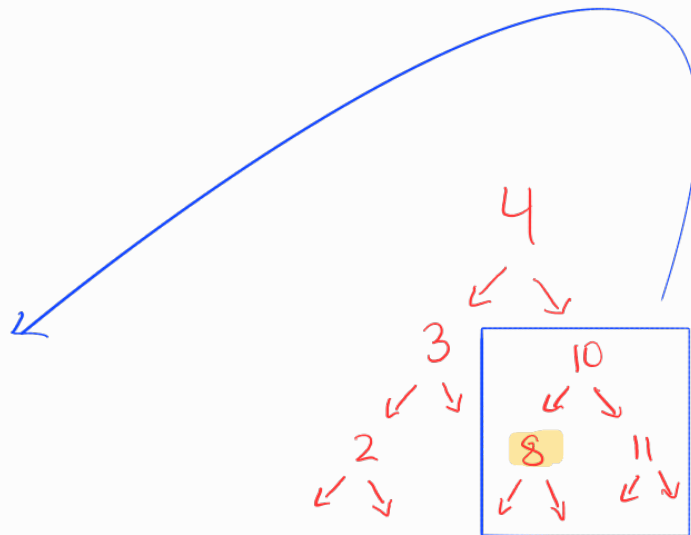
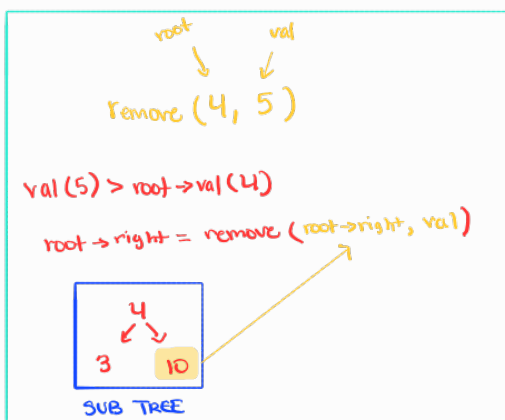
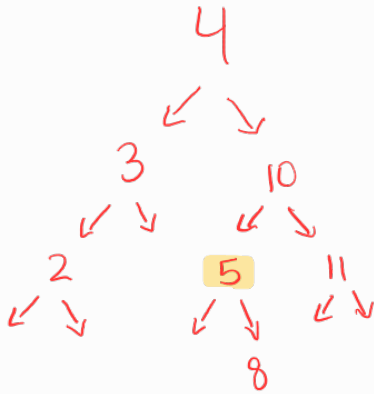
root → left = root → right (null)

- The text describes a recursive function to remove a node with a specific value (5) from a binary search tree.
- It traces the removal process, showing how the function navigates the tree based on value comparisons (5 vs. 4, 5 vs. 6), eventually finding the node to be removed.
- When the node (5) is found, it's identified as a leaf node, and its removal is handled by returning its right child (which is null in this case).

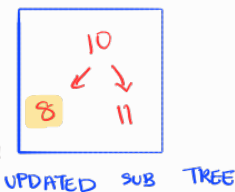
REMOVE CHILD w/ ONE LEAF NODE

TREE

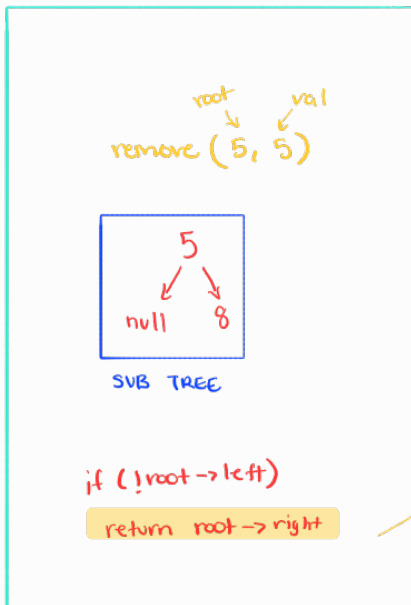
TARGET = 5



`root → left = root → right(8)`

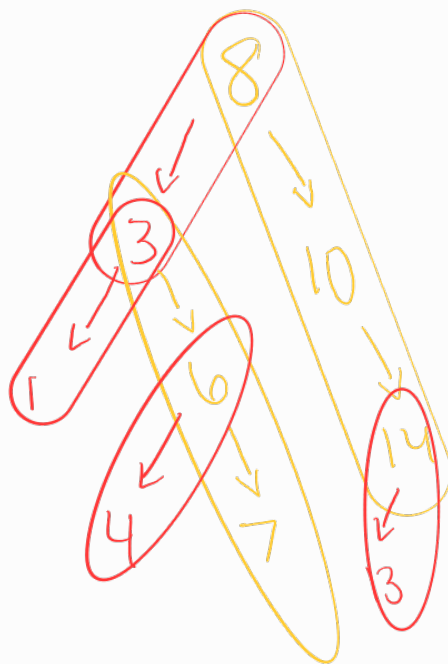
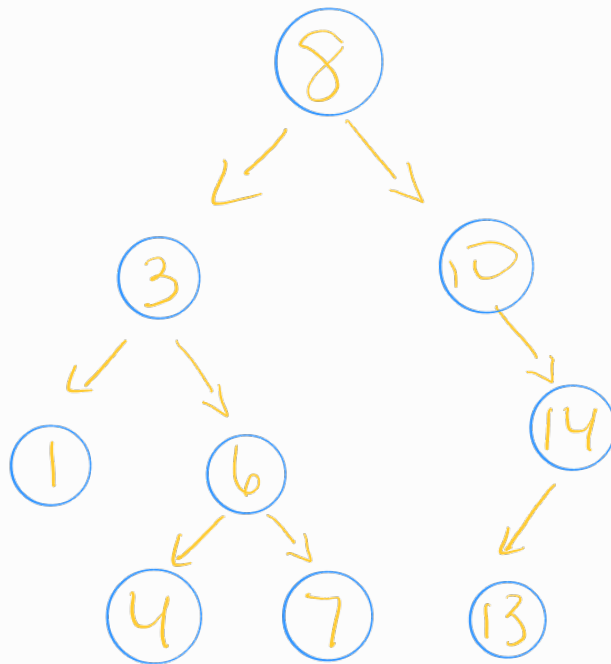


RETURN ROOT



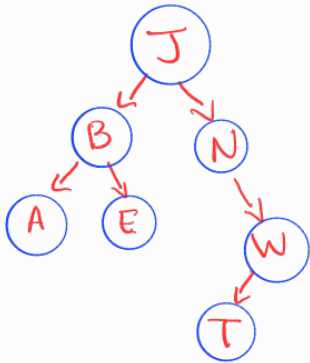
- The text describes a process for removing a child node with a single leak node from a tree.
- It illustrates this with examples of removing specific values (4, 5, 10) and shows how the `remove` function recursively updates the tree by adjusting `root->left` or `root->right` pointers.
- The process involves identifying the node to be removed and, if it has a single child, replacing it with that child.

Hardcode Binary Tree

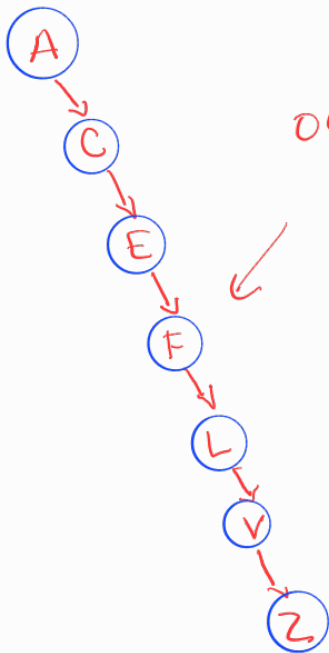


1. smallest to the left.
biggest to the right.

Jesus Morales

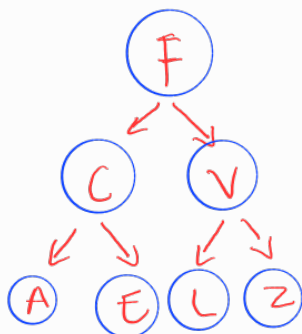


2.



$O(n)$ BST
↳ worst case
just a linked list

3. Example of a self balancing tree



3 levels instead of 7

$O(\log n)$ Avg case

F C V A E L Z